

DOKUMENTACJA TECHNICZNA

AirSense Weather AI

Opis każdego etapu systemu: źródła danych, czyszczenie, model i predykcja

Projekt	AirSense Weather AI — system predykcji jakości powietrza i pogody
Grupa	Grupa 5
Autorzy	Jakub Moszyński, Damian Tomczyk, Nikol Olszewska, Łukasz Duss
Technologie	Python, Streamlit, TensorFlow/Keras (LSTM), scikit-learn, Plotly
Źródła danych	GIOŚ API v1, Open-Meteo API
Data	3 czerwca 2026

1. Wprowadzenie

AirSense Weather AI to prototyp systemu uczenia maszynowego, który pobiera dane z rzeczywistych, publicznych API, przetwarza je i prognozuje krótkoterminowo (godzinowo) parametry pogodowe oraz jakości powietrza. Aplikacja zbudowana jest w Streamlit i obejmuje pełny cykl ML: pobieranie danych, preprocessing, analizę eksploracyjną (EDA), budowę i trening sieci LSTM, ewaluację oraz inferencję.

Niniejszy dokument opisuje krok po kroku każdy etap przetwarzania — od momentu pobrania surowych danych, przez ich czyszczenie, aż po architekturę modelu i sposób generowania prognozy.

2. Architektura i pipeline

System realizuje klasyczny pipeline ML w jednej aplikacji. Dane z dwóch źródeł są łączone w jeden zbiór dzienny, czyszczone, przekształcane w sekwencje czasowe i podawane do modelu LSTM, który generuje prognozę. Jakość modelu mierzona jest na wydzielonym zbiorze testowym.



Rys. 1. Pipeline przetwarzania danych — od API do prognozy.

3. Źródła danych

Aplikacja korzysta z dwóch niezależnych, darmowych API. Dla każdej z pięciu obsługiwanych stacji zapisany jest identyfikator GIOŚ oraz współrzędne geograficzne potrzebne do Open-Meteo.

3.1. Dane pogodowe — Open-Meteo

Z Open-Meteo pobierane są temperatura, wilgotność względna i prędkość wiatru jako dane **godzinowe** (~720 punktów na 30 dni). Model uczy się właśnie na tej rozdzielczości — agregacja do dni dawała tylko ~30 punktów, czyli zbiór zbyt mały do trenowania. Pobieramy 30 dni historii oraz bieżący dzień (**forecast_days=1**); dane są obcinane do bieżącej godziny, więc nie trafiają do nich godziny „w przyszłość”. Parametr **timezone=Europe/Warsaw** dopasowuje czas do strefy polskiej.

```
weather_url = (
    f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}"
    f"&past_days=30&forecast_days=1&timezone=Europe/Warsaw"
    f"&hourly=temperature_2m,relative_humidity_2m,wind_speed_10m"
)
df_weather = df_weather.set_index('Data').sort_index() # rozdzielczosc godzinowa
```

3.2. Dane o jakości powietrza — GIOŚ API v1

Z GIOŚ pobierana jest lista sensorów stacji, a następnie pomiary z każdego z nich. Serwer potrafi odrzucać zapytania bez nagłówków przeglądarki, dlatego dołączane są nagłówki HTTP (User-Agent, Referer, Accept). **Ważne:** API v1 zwraca klucze po polsku (np. „Wskaźnik - kod”, „Wartość”, „Identyfikator stanowiska”) opakowane w słownik — kod wykrywa je elastycznie (z fallbackiem do starszego, angielskiego API). Endpoint udostępnia tylko ostatnie ~3 dni pomiarów godzinowych.

```
headers = {
    'User-Agent': 'Mozilla/5.0 ... Chrome/122.0.0.0 Safari/537.36',
    'Accept': 'application/json, text/plain, */*',
    'Referer': 'https://powietrze.gios.gov.pl/'
}
url = f"https://api.gios.gov.pl/pjp-api/v1/rest/station/sensors/{station_id}"
s_res = requests.get(url, headers=headers, timeout=10)
```

3.3. Obsługiwane stacje pomiarowe

Stacja	ID GIOŚ	Szer. geogr.	Dł. geogr.
Zabrze	550	50.3124	18.7711
Warszawa — Marszałkowska	544	52.2287	21.0122
Kraków — Al. Krasińskiego	400	50.0574	19.9261
Gdańsk — Lektykarska	738	54.3497	18.6548
Wrocław — Korzeniowskiego	265	51.1294	17.0292

4. Czyszczenie i przygotowanie danych

Dane z rzeczywistych stacji nie są idealne — zawierają luki i pojedyncze błędne piki. Zastosowano następujące kroki czyszczenia:

- **Interpolacja liniowa** braków w obie strony (limit_direction='both').
- **Ograniczanie wartości odstających** metodą percentyli — przycinanie do 1. i 99. percentyla (winsoryzacja), co usuwa błędne piki czujników bez kasowania całych rekordów.
- **Łączenie źródeł** po dacie dziennej; brakujące kolumny pollutantów są dołączane dynamicznie.
- **Obcięcie do dnia dzisiejszego** — nie przechowujemy dni „w przyszłość”.

```
def remove_outliers(df):
    df_cleaned = df.copy()
    for col in df_cleaned.columns:
        if pd.api.types.is_numeric_dtype(df_cleaned[col]):
            lower = df_cleaned[col].quantile(0.01)
            upper = df_cleaned[col].quantile(0.99)
            df_cleaned[col] = np.clip(df_cleaned[col], lower, upper)
    return df_cleaned
```

Po interpolacji i usunięciu outlierów zbiór jest obcinany do bieżącej daty:

```
df_final = df_final.interpolate(method='linear', limit_direction='both')
df_final = remove_outliers(df_final)
df_final = df_final[df_final.index <= date.today()] # bez dni "w przyszłosc"
```

5. Sekwencje czasowe i skalowanie

Model uczy się przewidywać kolejne **godziny** na podstawie okna poprzednich godzin. Dane są najpierw skalowane do przedziału [0, 1] (MinMaxScaler), a następnie przekształcane w nakładające się sekwencje wejście→wyjście. Okno wejściowe (domyślnie 48 h) i horyzont prognozy (domyślnie 6 h) są parametrami konfigurowalnymi w aplikacji.

```
def prepare_sequences(data, time_steps, forecast_horizon):
    X, y = [], []
    for i in range(len(data) - time_steps - forecast_horizon + 1):
        X.append(data[i:(i + time_steps), :])
        y.append(data[(i + time_steps):(i + time_steps + forecast_horizon), :])
    return np.array(X), np.array(y)
```

6. Model LSTM (Sequence-to-Sequence)

Sercem systemu jest dwuwarstwowa sieć LSTM typu sequence-to-sequence z regularyzacją Dropout. Model przyjmuje okno cech z kilku dni i zwraca prognozę wszystkich cech na kilka dni naprzód. Liczba cech jest dynamiczna — dopasowana do liczby aktywnych sensorów danej stacji.



Rys. 2. Architektura modelu LSTM Seq2Seq.

```
model = Sequential([
    LSTM(lstm_units, return_sequences=True, input_shape=(time_steps, N_FEATURES)),
    Dropout(0.2),
    LSTM(lstm_units, return_sequences=False),
    Dropout(0.2),
    Dense(forecast_horizon * N_FEATURES),
    Reshape((forecast_horizon, N_FEATURES)),
])
model.compile(optimizer='adam', loss='mse')
```

Hiperparametry i podział zbioru

Parametr	Wartość
Okno wejściowe (time_steps)	12-96 h (domyślnie 48)
Horyzont prognozy (forecast_horizon)	1-24 h (domyślnie 6)
Liczba neuronów LSTM	50-100 (domyślnie 64)
Liczba epok	do 150 + early stopping (patience 12)
Batch size	16
Dropout	0.2
Optymalizator / funkcja straty	Adam / MSE
Podział zbioru (chronologiczny)	80% trening / 10% walidacja / 10% test

Podział jest **chronologiczny** (bez tasowania) — to istotne dla szeregów czasowych, aby model nie „widział” przyszłości podczas treningu:

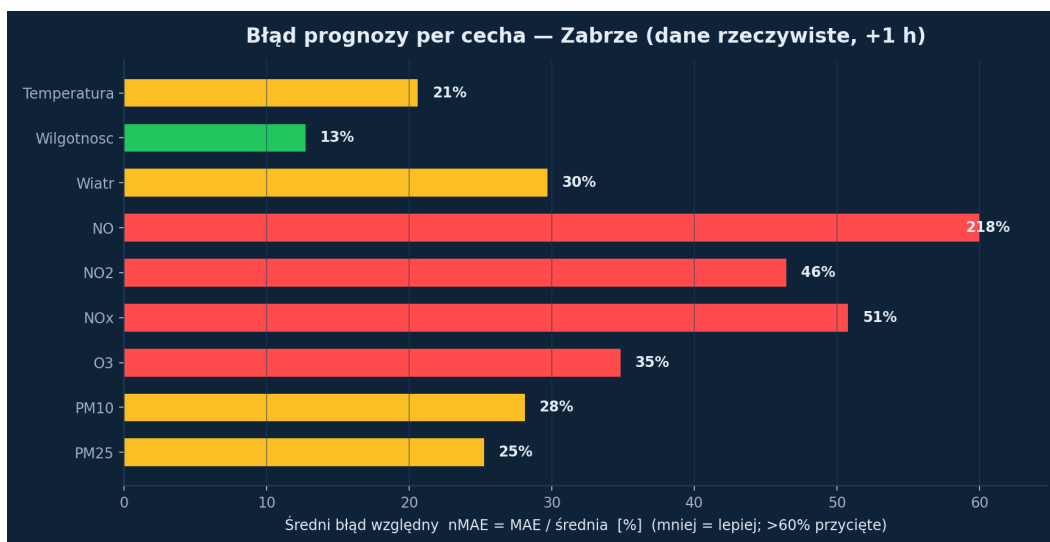
```
total = len(X)
train_len = int(total * 0.8)
val_len = int(total * 0.1)
X_train, y_train = X[:train_len], y[:train_len]
X_val, y_val = X[train_len:train_len+val_len], y[train_len:train_len+val_len]
X_test, y_test = X[train_len+val_len:], y[train_len+val_len:]
```

7. Ewaluacja modelu

Ewaluacja odbywa się na 10% zbiorze testowym, którego model nie widział podczas treningu. Metryki liczone są w **jednostkach rzeczywistych** — predykcje i wartości oczekiwane są najpierw odskalowane (inverse_transform), a dopiero potem porównywane. Dzięki temu MAE i RMSE są interpretowalne (np. stopnie, $\mu\text{g}/\text{m}^3$), a nie wyrażone w bezwymiarowej skali [0, 1].

```
y_test_real = scaler.inverse_transform(y_test.reshape(-1, N_FEATURES))
y_pred_real = scaler.inverse_transform(y_pred.reshape(-1, N_FEATURES))
mae = mean_absolute_error(y_test_real, y_pred_real)
rmse = np.sqrt(mean_squared_error(y_test_real, y_pred_real))
r2 = r2_score(y_test_real, y_pred_real) # uśrednione po cechach
```

Metryki liczone są też w rozbiciu na cechy. Poniższy wykres pokazuje realny wynik (stacja Zabrze, dane godzinowe, prognoza +1 h) jako średni błąd względny $\text{nMAE} = \text{MAE} / \text{średnia}$.



Rys. 3. Realny błąd prognozy (nMAE) w rozbiściu na cechy — stacja Zabrze.

Realne metryki (zbiór testowy, jednostki rzeczywiste)

Cecha	MAE	R ²
Ogółem	6.22	−0.25
Temperatura	3.18 °C	−1.31
Wilgotność	8.54 %	0.43
Wiatr	2.57	0.11
PM10	3.91 µg/m ³	−0.25
PM2.5	2.78 µg/m ³	−0.64
O3	18.9 µg/m ³	−0.01

Interpretacja (uczciwa): MAE jest stabilne i interpretowalne — np. temperatura myli się średnio o $\sim 3^{\circ}\text{C}$. R^2 jest jednak bliskie zera, a często ujemne i niestabilne między uruchomieniami, co oznacza, że model jest na poziomie prostego baseline'u. Przyczyny: bardzo mały zbiór testowy (~ 3 dni) oraz dane GIOŚ o zanieczyszczeniach dostępne tylko za ~ 3 dni (reszta okna jest interpolowana). Jest to działający prototyp i kompletny pipeline, ale jeszcze nie dokładny predyktor — najlepiej radzi sobie z krótkoterminową prognozą pogody (wilgotność, temperatura). Główny kierunek rozwoju: więcej danych (dłuższa historia, archiwalne dane GIOŚ).

8. Predykcja i inferencja

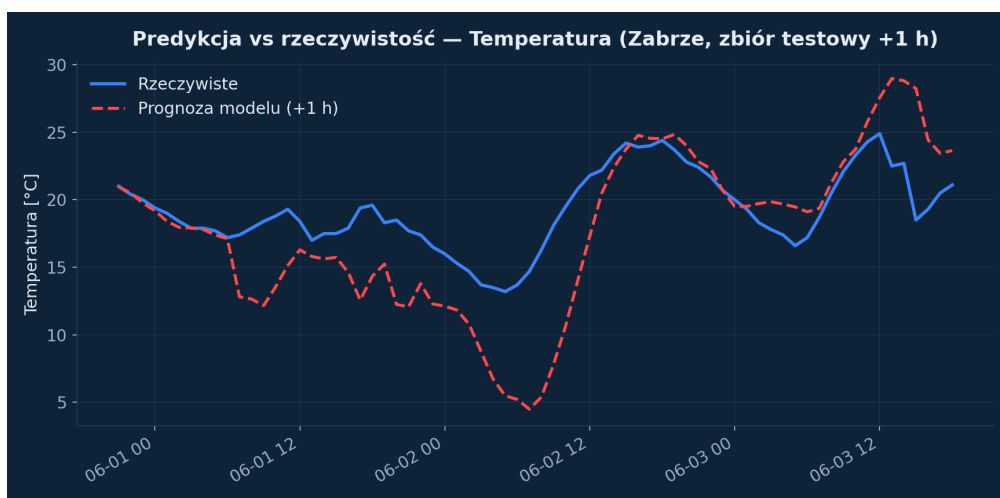
W zakładce predykcji użytkownik wybiera zapisany model, a aplikacja odtwarza parametry skalera z metadanych. Macierz wejściowa (ostatnie dni) jest edytowalna. Przy odtwarzaniu skalera zabezpieczono się przed cechami stałymi — bez tego dzielenie przez zero dawałoby prognozę NaN:

```
data_range = scaler_max - scaler_min
data_range[data_range == 0] = 1.0 # cechy stałe -> brak dzielenia przez zero
trained_scaler.scale_ = 1.0 / data_range
trained_scaler.min_ = -scaler_min * trained_scaler.scale_
```

Prognoza zaczyna się godzinę po ostatnim realnym pomiarze, a oś czasu na wykresie to prawdziwe znaczniki czasu:

```
last_date = pd.Timestamp(df_live.index[-1])
future_dates = [last_date + timedelta(hours=i) for i in range(1, f_horizon + 1)]
```

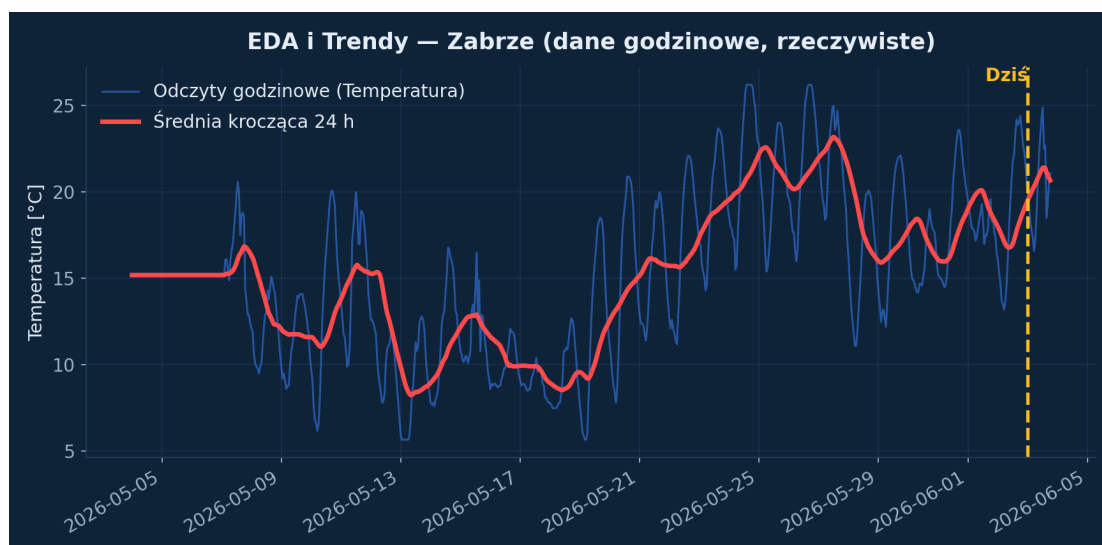
Do oceny jakości najlepiej posłużyć się wykresem predykcja vs rzeczywistość na zbiorze testowym (poniżej, dla temperatury, prognoza +1 h) — model śledzi ogólny kształt, ale z wyraźnymi błędami.



Rys. 4. Predykcja vs rzeczywistość — temperatura, zbiór testowy (+1 h).

9. Dashboard i analiza eksploracyjna (EDA)

Pierwsza zakładka prezentuje surowe odczyty godzinowe wraz z 24-godzinną średnią kroczącą (trend dobowy). Trend liczony jest jako osobna seria, aby nie zmieniać liczby cech przekazywanych do modelu. Pionowa linia „Dziś” wyraźnie oddziela dane historyczne od przyszłości (za którą odpowiada już prognoza).



10. Ostatnie usprawnienia jakości

- **Naprawiono parser GIOŚ v1** (polskie klucze) — wcześniej żaden wskaźnik zanieczyszczeń się nie wczytywał i widoczna była tylko pogoda.
- **Przejęcie na modelowanie godzinowe** zamiast dziennego — realnie trenowalny zbiór (~720 zamiast ~30 punktów).
- Dodano early stopping — model bywał niedouczony przy małej liczbie epok.
- Dodano pionową linię „Dziś” na wykresach EDA i predykcji.
- Wyłączono pobieranie godzin „w przyszłość” (forecast_days=1 + obcięcie do teraz).
- Metryki liczone w jednostkach rzeczywistych oraz w rozbiciu na cechy.
- Zabezpieczenie odtwarzania skalera przed cechami stałymi (brak prognoz NaN);
.bfill().ffill()).

11. Uruchomienie

```
pip install -r requirements.txt
streamlit run app.py
```

Aplikacja uruchamia się w przeglądarce. W panelu bocznym wybiera się stację; w zakładkach kolejno: analiza EDA, trening modelu i predykcja.

12. Struktura plików

Plik / katalog	Znaczenie
app.py	Główna aplikacja Streamlit: EDA, trening LSTM, ewaluacja, predykcja.
train_model.py	Pomocniczy skrypt treningowy (podstawowy LSTM na danych AQI).
requirements.txt	Lista zależności (Streamlit, TensorFlow, scikit-learn, Plotly...).
airsense_lstm.keras	Zapisany przykładowy model Keras.
metrics.npy, scaler_params.npy	Metryki i parametry skalera przykładowego modelu.
models/	Modele wytrenowane w aplikacji (.keras + _meta.npy).
dokumenty/	Dokumentacja, raport, prezentacja i wykresy.

13. Ograniczenia i dalszy rozwój

- Dane GIOŚ o zanieczyszczeniach dostępne tylko za ~3 dni → pollutanty są w dużej części interpolowane i słabo przewidywane ($R^2 \leq 0$).

- Mały zbiór testowy → R^2 niestabilne (bliskie 0); model jest na poziomie baseline'u. Najlepiej działa krótkoterminowa prognoza pogody (wilgotność, temperatura, błąd ~13-21%).
- Kierunki rozwoju: archiwalne dane GIOŚ (dłuższa historia zanieczyszczeń), więcej miast i automatyczna detekcja stacji, modelowanie pojedynczych cech lub ważenie strat, porównanie z GRU / Prophet / XGBoost / Transformerami, automatyczny re-trening i baza historii predykcji.